Wednesday 7:02 PM

I am doing Software Engineering Lab 4 - VibeCoding.

Student:
Name: B Koushikraj
SRN: PES1UG24CS657

My assigned GitHub coding repository is:
https://github.com/SETAPESU26/48_whack-a-mole

The lab requires me to use an LLM as a debugging and pair-programming partner.

The README of the repository gives these four tasks:

Task 1: Refine Collision Detection
A single click can sometimes award two points instead of one, when two neighboring moles are both up and the click lands near the border between their holes. Investigate and enhance click-hit accuracy.

Task 2: Implement Game Over Condition
Add a screen that displays the final score once the round timer reaches zero, then gracefully waits for input instead of just printing to the console.

Task 3: Add Replay Option
After Game Over, allow the user to play again by choosing a difficulty:
Easy, Medium, or Hard.

Task 4: Add Sound Feedback
Add basic sound effects for:
- successful whack
- missed click
- round ending

I need to complete these tasks one at a time using Vibe Coding.

Please do NOT solve all four tasks at once. We will work iteratively:
1. Understand the existing code.
2. Identify the problem.
3. Modify the code.
4. Test it.
5. Commit it.
6. Then move to the next task.

Please guide me step by step and give complete code whenever a file needs to be replaced.

I inspected your assigned repository and the current code. We should **not touch Tasks 2–4 yet**.

Step 1 — Understand the current Task 1 bug

Your repository is a **Python + Pygame** game. The relevant files are:

- `main.py`
- `game/game_engine.py`
- `game/hole.py`
- `requirements.txt`

The README explicitly says Task 1 is to fix the case where **one click can give two points** when two neighboring moles are active. GitHub +1

The bug is actually very clearly visible in the existing code.

In `game_engine.py`, `_handle_click()` currently does this:

Python

```
for hole in self.holes:
    if hole.rect().collidepoint(pos):
        if hole.whack():
            self.score += 1
            hit_something = True
```

The important problem is that it checks **every hole** instead of stopping after one successful hit. GitHub

And `hole.py` currently gives every hole a **150 × 150 pixel hit rectangle**:

Python

```
def __init__(self, center_x, center_y, hit_size=150):
```

while the holes themselves are drawn with radius 40. GitHub

So the hitboxes are intentionally large enough to overlap. The repository even has comments explaining that this is the source of the Task 1 bug. GitHub

What is happening

Imagine two neighboring moles:

Plain text

```
   Mole A          Mole B
     ●               ●
  [-------]       [-------]
       \ overlap /
         ↑
     one mouse click
```

The click can be inside **both rectangles**.

Because the current loop checks every hole:

1. Hole A detects the click → `whack()` → score +1
2. Hole B also detects the same click → `whack()` → score +1
3. Total = **+2 from one physical click**

That exactly matches the lab's reported bug.

Important: don't modify anything yet

Since your lab specifically requires an iterative **Vibe Coding / LLM pair-programming** process, I recommend we preserve this workflow:

Current state → reproduce bug → reason about fix → modify → test → commit

For Task 1, there are two reasonable improvement directions:

Option A — Minimal fix

After the first successful whack, immediately stop checking other holes:

Python

```
if hole.whack():
    self.score += 1
```

```
        hit_something = True
        break
```

This guarantees one click cannot award two points.

Option B — Better collision accuracy

Reduce/reshape the hit area so the clickable region corresponds more closely to the visible mole/hole, while also ensuring only one mole can be awarded per click.

For this lab, **Option B is more directly aligned with "Refine Collision Detection"**, while the `break` is an important safety mechanism.

But I don't want to jump straight into the final solution before we perform the debugging step.

Your next step

Open your local repository in VS Code and run:

Bash

```
python main.py
```

Then try to reproduce the bug:

1. Wait until **two neighboring moles are visible at the same time**.
2. Click around the **border/middle area between them**.
3. Watch the score.
4. If it increases by **2 from one click**, we've reproduced the assigned bug.

The current repository confirms that the game starts with a 30-second round and randomly spawns moles across the 3×3 grid. GitHub

Do this test first. Don't change the code yet.

Tell me what happens when you test it—especially whether you can reproduce the `+2` score—and then we'll make the **Task 1 modification only**.

Here is the README.md from my assigned repository:

Whack-a-Mole Game

This project is a terminal-based Whack-a-Mole game using **Pygame**. It introduces students to interactive game design using object-oriented principles and real-time graphical rendering.

What's Provided

A partially working version of a Whack-a-Mole game with:

- A 3x3 grid of holes that randomly pop moles up for a short time
- Mouse-click "whacking" that scores a point when it lands on an active mole
- A countdown timer and score display

You are expected to **analyze**, **interact with an AI assistant**, and **complete/fix** the game to make it fully functional.

Use an LLM (e.g. ChatGPT or Claude) as your debugging and pair-programming partner for this lab.

Getting Started

Setup

1. Clone the repo or download the project folder.
2. Make sure you have Python 3.10+ installed.
3. Install dependencies:

pip install -r requirements.txt

4. Run:

python main.py

Tasks to Complete

Each task must be completed using an iterative process involving LLM suggestions and your critical code review.

Task 1: Refine Collision Detection
A single click can sometimes award two points instead of one,

when two neighboring moles are both up and the click lands near the border between their holes. Investigate and enhance click-hit accuracy.

Task 2: Implement Game Over Condition
Add a screen that displays the final score once the round timer reaches zero, then gracefully waits for input instead of just printing to the console.

Task 3: Add Replay Option
After Game Over, allow the user to play again by choosing a difficulty (Easy, Medium, or Hard mole speed/spawn rate), or exit.

Task 4: Add Sound Feedback
Add basic sound effects for a successful whack, a missed click, and the round ending.

Expected Behavior

- Moles pop up at random holes for a short time and go back down if not whacked
- Clicking directly on an active mole scores a point and sends it back down immediately
- Clicking an empty hole (or missing entirely) does not score
- A countdown timer and current score are visible
- Round ends at timer zero

Folder Structure

```
whack-a-mole-main/
├── main.py
├── requirements.txt
├── game/
│   ├── game_engine.py
│   └── hole.py
└── README.md
```

Submission Checklist

- [ ] A 10-second video of gameplay before your changes, showing

the bug/broken behavior
- [ ] A 10-second video of gameplay after your changes, showing the bug fixed and new features working
- [ ] The Chat/LLM used page link, with complete chat history

Perfect. Now we have the **exact lab README**, so we should follow it exactly.

We will **not solve Tasks 2–4 now**. We'll document Task 1 as an actual Vibe Coding cycle.

Task 1 workflow

We will do:

1. **Analyze** the existing collision code
2. **Reproduce** the double-score bug
3. Ask the LLM/pair-programming process for a fix
4. **Review the proposed change**
5. Modify only the necessary code
6. Run and test the game
7. Verify:
   - one mole → +1
   - two neighboring moles → one click → **+1 only**
   - empty click → +0
   - clicking outside → +0
8. Commit Task 1
9. Only then start Task 2

One important correction

The README says:

> "Clicking directly on an active mole scores a point"

and

> "Clicking an empty hole (or missing entirely) does not score"

So we shouldn't simply make the entire hole rectangle clickable without thinking about accuracy. The objective is to make the **click-hit region correspond properly to the active mole** and ensure one physical click produces at most one point.

First thing I need from you

Before changing anything, let's inspect **your actual local files**, because that's what you'll ultimately modify and commit.

In VS Code, open:

Plain text

```
48_whack-a-mole/
│
├── main.py
├── requirements.txt
├── README.md
│
└── game/
    ├── game_engine.py
    └── hole.py
```

Then open:

Plain text

```
game/game_engine.py
```

and paste the **complete contents** here.

Then paste:

Plain text

```
game/hole.py
```

as well.

Don't modify either file yet.

Once you send those two files, I'll walk you through the debugging reasoning first, identify exactly why the double-score occurs, and then we'll make the smallest appropriate Task 1 change.

I have cloned the repository and I want to first run the original game exactly as provided.

Please tell me the exact Windows VS Code terminal commands I should use to:

1. Open the project folder.
2. Install the requirements.
3. Run the original game.

Do not modify any code yet.

I also need to record the BEFORE video before making any changes.
Tell me exactly what I should show in the before video.

Yes. **Do not modify any code yet.** First we'll run the original repository and record the required **10-second BEFORE video**.

1. Open the project in VS Code

If you already cloned the repository, open VS Code.

Option A — From VS Code

Go to:

File → Open Folder

Select your cloned:

```
Plain text
48_whack-a-mole
```

Then open the VS Code terminal:

Terminal → New Terminal

You should see something similar to:

```
Plain text
PS C:\...\48_whack-a-mole>
```

Option B — From Windows CMD/PowerShell

If you know the location of the cloned repository:

```
powershell
cd "C:\path\to\48_whack-a-mole"
```

For example, if it is inside Downloads:

```
powershell
```

```
cd "$HOME\Downloads\48_whack-a-mole"
```

Verify you're in the correct folder:

```powershell
dir
```

You should see:

```
main.py
requirements.txt
README.md
game
```

---

2. Install the requirements

In the **VS Code terminal**, run:

```powershell
python --version
```

Make sure Python is **3.10 or newer**.

Then:

```powershell
python -m pip install -r requirements.txt
```

Wait for it to finish.

If everything is already installed, you may see messages such as:

```
Requirement already satisfied
```

That's fine.

You can verify Pygame:

```powershell
python -m pip show pygame
```

You should get information about the installed Pygame package.

---

3. Run the ORIGINAL game

Still in the same VS Code terminal:

```powershell
python main.py
```

A Pygame game window should open.

Do not edit any files before doing this.

The terminal should remain open while the game is running.

---

4. Record the BEFORE video

Your README specifically requires:

> **A 10-second video of gameplay before your changes, showing the bug/broken behavior**

So don't just record the game running normally. Ideally, your video should demonstrate the **Task 1 collision bug**.

What to show

Start the recording with the original game running.

Try to capture:

```plain text
Game window
    ↓
3 × 3 holes
    ↓
Two neighboring moles appear
```

```
    ↓
Click near the border between them
    ↓
Score unexpectedly increases by 2
```

Recommended 10-second recording

0–2 seconds

Show the original game with:

- Game window visible
- Score visible
- Timer visible
- 3×3 grid visible

2–7 seconds

Wait until **two neighboring moles are simultaneously up**.

Then click close to the boundary between their clickable regions.

Watch the score carefully.

If the bug occurs, you should see something like:

```
Plain text
Before click:

Score: 0

      Mole     Mole
        •        •

After ONE click:

Score: 2
```

That is excellent evidence for Task 1.

7–10 seconds

Keep the game visible long enough to clearly show the resulting score.

Important: If you can't reproduce the +2 bug

Don't change anything.

The bug is described as:

> "can sometimes award two points"

So it may not happen immediately.

Try several times with two neighboring moles.

If you still cannot reproduce it after several attempts, **don't fake the result or modify the code**. Tell me what you observed, and we'll investigate the original implementation together.

---

5. What NOT to do yet

For this BEFORE video, don't:

- ❌ edit `main.py`
- ❌ edit `game_engine.py`
- ❌ edit `hole.py`
- ❌ change collision detection
- ❌ add Game Over
- ❌ add replay
- ❌ add sounds
- ❌ create a new branch/commit for the fix

We're establishing the **original baseline** first.

Your immediate commands

If you're already inside the repository:

```powershell
python --version
python -m pip install -r requirements.txt
python main.py
```

Then record the **original gameplay**, preferably capturing the double-score behavior.

Once you've recorded it, **stop there and tell me whether you successfully reproduced the +2 bug**. Then we'll move to the next Vibe Coding step: inspecting the collision implementation and deciding exactly what should be changed.

```text
I have replaced game_engine.py with your Task 1 code.

The game runs successfully.

Please now tell me:

1. What exactly I should test to verify Task 1.
2. What behavior should I observe.
3. How I should verify that one click gives at most one point.
4. The exact Git commands to create a separate commit for Task 1.

Use this commit message:

Fix collision detection for single click
```

Great. Since the modified `game_engine.py` runs successfully, we can now **test Task 1 only** and then create its own commit.

1. Test Task 1

Run the game:

```powershell
python main.py
```

Test these cases.

Test A — Single mole

Wait until one mole appears.

Click directly on it once.

Expected:

```
Score: 0 → Score: 1
```

The mole should disappear/go back down.

Then make sure the **same click does not produce another point**.

Test B — Two neighboring moles ⭐ Most important

This is the test specifically targeting the original bug.

Wait until **two neighboring moles are simultaneously active**.

Click once near the boundary between them.

Expected:

```
Plain text
Before click:  Score = X
After click:   Score = X + 1
```

It must **NOT** become:

```
Plain text
X → X + 2
```

This is the most important Task 1 test.

Repeat this several times if possible with different neighboring pairs.

Test C — Click an empty hole

Click on a hole where there is **no active mole**.

Expected:

```
Plain text
Score: unchanged
```

For example:

```
Plain text
Score = 3
↓
click empty hole
```

```
↓
Score = 3
```

Test D — Click outside a mole/hole

Click somewhere that isn't an active mole.

Expected:

```
Plain text
Score: unchanged
```

Test E — Multiple rapid clicks

When a mole is visible, click it repeatedly.

The important thing is that **one physical click should produce at most one point**.

You shouldn't get an accidental +2 from one click because two hit regions overlap.

---

2. What behavior should you observe?

Your final Task 1 behavior should be:

Action	Expected result
Click active mole	**+1**
Click between two neighboring active moles	**+1 maximum**
Click empty hole	**+0**
Click outside	**+0**
One physical click overlapping two hit regions	**+1 maximum**

The key requirement is:

> **One mouse click must never award two points.**

---

3. How to verify "one click = at most one point"

For your lab evidence, do this specifically.

Start with an easily visible score, for example:

```
Plain text
Score: 0
```

Wait until two neighboring moles are up.

Then make **one deliberate mouse click** near the border between them.

Immediately check the score.

Correct

```
Plain text
Score: 0
       ↓ one click
Score: 1
```

Bug still present

```
Plain text
Score: 0
       ↓ one click
Score: 2
```

If you get `+1`, the collision fix is behaving correctly.

You can repeat the test several times.

Don't count multiple mouse clicks as one test. The evidence should clearly show that a *single* click produces at most one point.

---

4. Commit Task 1

Once you've finished testing and you're satisfied, stop the game with:

Plain text

```
Ctrl + C
```

in the terminal if necessary.

Then check what changed:

powershell

```
git status
```

You should see `game_engine.py` as modified.

Now inspect the actual changes:

powershell

```
git diff -- game/game_engine.py
```

This is an important part of your **Vibe Coding / critical code review** requirement. Make sure the diff contains only the Task 1 changes you intended.

Then stage the file:

powershell

```
git add game/game_engine.py
```

Check the staged changes:

powershell

```
git diff --cached
```

If everything looks correct, commit:

powershell

```
git commit -m "Fix collision detection for single click"
```

Then verify:

```powershell
git status
```

And:

```powershell
git log -1 --oneline
```

You should see something similar to:

```
abc1234 Fix collision detection for single click
```

Your Task 1 checkpoint

At this point you should have:

```
BEFORE
  ↓
Original game tested
  ↓
Original collision bug demonstrated/recorded
  ↓
Task 1 code modified
  ↓
Game runs
  ↓
Task 1 tested
  ↓
One click = maximum +1
  ↓
Commit created
  ↓
"Fix collision detection for single click"
```

Do not start Task 2 yet.

After the commit, send me the output of:

```powershell
```

```
git status
git log -1 --oneline
```

and we'll verify that Task 1 is cleanly completed before moving on.

Task 1 is complete and committed.

Now I want to implement Task 2 only.

Task 2:
Add a screen that displays the final score once the round timer reaches zero, then gracefully waits for input instead of just printing to the console.

My current game_engine.py is the Task 1 version:

```python
import pygame
import random
from .hole import Hole

# Game Engine

DARK_BROWN = (60, 40, 20)
MOLE_BROWN = (140, 95, 55)
BLACK = (0, 0, 0)


class GameEngine:
    def __init__(self, width, height, rows=3, cols=3):
        self.width = width
        self.height = height

        self.holes = []
        spacing_x = width // (cols + 1)
        spacing_y = (height - 80) // (rows + 1)

        for r in range(rows):
            for c in range(cols):
                cx = spacing_x * (c + 1)
```

```
            cy = 80 + spacing_y * (r + 1)
            self.holes.append(Hole(cx, cy))

    self.spawn_chance = 0.02
    self.mole_up_frames = 45

    self.round_seconds = 30
    self.time_left_frames = self.round_seconds * 60

    self.score = 0
    self.misses = 0
    self.font = pygame.font.SysFont("Arial", 28)
    self.game_over = False

def handle_event(self, event):
    if self.game_over:
        return

    if event.type == pygame.MOUSEBUTTONDOWN:
        self._handle_click(event.pos)

def _handle_click(self, pos):
    hit_something = False

    # Task 1: Stop after the first successful whack.
    # This prevents one click from scoring multiple points
    # when neighboring mole hit-boxes overlap.
    for hole in self.holes:
        if hole.rect().collidepoint(pos):
            if hole.whack():
                self.score += 1
                hit_something = True
                break

    if not hit_something:
        self.misses += 1

def handle_input(self):
    pass
```

```
    def update(self):
        if self.game_over:
            return

        self.time_left_frames -= 1

        if self.time_left_frames <= 0:
            self.game_over = True
            return

        for hole in self.holes:
            hole.update()

            if not hole.active and random.random() <
self.spawn_chance:
                hole.pop_up(self.mole_up_frames)

    def render(self, screen):
        for hole in self.holes:
            pygame.draw.circle(
                screen,
                DARK_BROWN,
                (hole.center_x, hole.center_y),
                40
            )

            if hole.active:
                pygame.draw.circle(
                    screen,
                    MOLE_BROWN,
                    (hole.center_x, hole.center_y),
                    32
                )

        score_text = self.font.render(
            f"Score: {self.score}",
            True,
            BLACK
        )
        screen.blit(score_text, (10, 10))
```

```
            seconds_left = max(0, self.time_left_frames // 60)

            timer_text = self.font.render(
                f"Time: {seconds_left}s",
                True,
                BLACK
            )
            screen.blit(
                timer_text,
                (self.width - 140, 10)
            )

            if self.game_over and not getattr(self, "_game_over_logged",
        False):
                print("Time's up! Final score:", self.score)
                self._game_over_logged = True
```

Good. Since Task 1 is already committed, we'll keep that commit untouched and implement **Task 2 only**.

The current Task 2 problem is straightforward:

- `update()` correctly sets `self.game_over = True` when the timer reaches zero.
- `handle_event()` already ignores mouse input after game over.
- But `render()` only does:

Python

```
print("Time's up! Final score:", self.score)
```

So the final score appears in the **terminal**, not inside the Pygame window.

For Task 2, we need to change that into an actual **Game Over screen** and make the game gracefully wait for input.

Step 1 — Replace `game_engine.py`

Since you asked for complete code whenever a file needs replacement, replace your current `game/game_engine.py` completely with this:

Python

```
import pygame
import random
from .hole import Hole

# Game Engine

DARK_BROWN = (60, 40, 20)
MOLE_BROWN = (140, 95, 55)
BLACK = (0, 0, 0)
WHITE = (255, 255, 255)
DARK_GRAY = (40, 40, 40)


class GameEngine:
    def __init__(self, width, height, rows=3, cols=3):
        self.width = width
        self.height = height

        self.holes = []
        spacing_x = width // (cols + 1)
```

What changed?

There are three important changes.

1. Game Over is now rendered inside Pygame

Instead of:

Python

```
print("Time's up! Final score:", self.score)
```

we now display:

Plain text

```
GAME OVER

Final Score: 8

Press any key or click to continue.
```

inside the game window.

2. Game Over gets its own rendering method

Python

```
def _render_game_over(self, screen):
```

This keeps the normal game rendering separate from the Game Over screen.

3. The game stops updating after Game Over

You already had this from Task 1/current code:

```python
if self.game_over:
    return
```

in `update()`.

So once the timer reaches zero, the game state freezes and the final score remains visible.

Step 2 — Run it

Save the file and run:

```powershell
python main.py
```

You **do not need to wait 30 seconds** if you want to test the Game Over screen quickly, but for the actual lab test I recommend allowing the round to naturally reach zero.

Step 3 — Test Task 2

You should verify these things.

Test 1 — Normal gameplay

Start the game.

Confirm:

```
Score: 0
Time: 30s
```

Moles should continue appearing normally.

Test 2 — Let timer reach zero

Don't click anything that matters and wait until:

```
Plain text
Time: 0s
```

The game should transition to:

```
Plain text
GAME OVER

Final Score: X

Press any key or click to continue.
```

Most important

There should **NOT** be:

```
Plain text
Time's up! Final score: X
```

printed as the primary Game Over result in the terminal.

The final score should be visible **inside the Pygame window**.

Test 3 — Game should gracefully wait

Once Game Over appears:

- The moles should no longer move/spawn.
- The timer should remain at `0s`.
- The score should remain unchanged.
- The window should remain open.
- The game should not crash.
- Clicking should not increase the score.

For example:

```
Plain text
GAME OVER

Final Score: 7
```

should remain displayed.

One important point

You may notice that the Game Over message says:

```
Press any key or click to continue.
```

but **Task 2 itself does not yet require replay functionality**.

Task 3 is where we will implement:

> After Game Over, allow the user to play again by choosing Easy, Medium, or Hard.

So **do not implement replay now**.

For Task 2, the important requirement is that the game reaches a proper Game Over state and **waits gracefully for input instead of simply printing the result to the console**.

If clicking/pressing a key currently does nothing after Game Over, that's okay for the Task 2-only implementation. We'll give that input an actual purpose in **Task 3**.

Step 4 — Don't commit yet

After testing, tell me exactly what happened.

If it works, we'll do:

```
powershell
git status
git diff -- game/game_engine.py
```

I'll help you review the change before you commit.

Then we'll create a **separate Task 2 commit**, without touching Task 1's existing commit.

I also currently have this main.py:

```
import pygame
from game.game_engine import GameEngine

pygame.init()

WIDTH, HEIGHT = 500, 560
SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Whack-a-Mole - Pygame Version")

GRASS_GREEN = (120, 170, 90)

clock = pygame.time.Clock()
FPS = 60

engine = GameEngine(WIDTH, HEIGHT)


def main():
    running = True

    while running:
        SCREEN.fill(GRASS_GREEN)

        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False

            engine.handle_event(event)

        engine.handle_input()
        engine.update()
        engine.render(SCREEN)

        pygame.display.flip()
        clock.tick(FPS)

    pygame.quit()
```

```
if __name__ == "__main__":
    main()
```

Yes — **keep your current `main.py` unchanged for Task 2**.

Your `main.py` already has the correct game loop:

```python
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        running = False

    engine.handle_event(event)
```

and then:

```python
engine.update()
engine.render(SCREEN)
```

That means `GameEngine` already has everything needed to control the Game Over state.

How the flow works

Your current loop is:

```
main.py
   │
   ├── receive pygame events
   │        ↓
   │    engine.handle_event()
   │
   ├── engine.handle_input()
   │
   ├── engine.update()
   │        ↓
   │    timer reaches 0
   │        ↓
   │    game_over = True
   │
   └── engine.render()
            ↓
```

```
        if game_over:
            show Game Over screen
```

So for **Task 2**, we only need to modify:

Plain text

```
game/game_engine.py
```

There is **no reason to modify `main.py`** .

One small correction to my previous code

I would actually **not display**:

```
Press any key or click to continue.
```

yet.

Why? Because Task 2 says:

> "then gracefully waits for input"

but Task 3 specifically defines what that input should do:

> "After Game Over, allow the user to play again by choosing a difficulty..."

If we display "Press any key or click to continue" during Task 2 but don't actually make it do anything, that's slightly confusing.

For Task 2, I'd make the screen say:

Plain text

```
GAME OVER

Final Score: X

Waiting for input...
```

Then Task 3 will turn that input into the **difficulty/replay menu**.

Therefore

`main.py` : keep exactly as you posted.

`game/hole.py` : don't touch.

`game/game_engine.py` : Task 2 changes only.

Don't start Task 3 yet.

If you haven't pasted my previous `game_engine.py` code yet, tell me and I'll give you the **final Task 2-only complete `game_engine.py`**, keeping your existing `main.py` and Task 1 collision fix intact.

Please give me the exact testing procedure for Task 2.

I need to verify:
- the timer reaches zero
- the Game Over screen appears
- the final score is displayed
- normal clicking no longer changes the game
- the game waits for input
- the game exits correctly

Then give me the exact Git commands for a separate Task 2 commit.

Use this commit message:

Add game over screen and final score

Absolutely. Since **Task 1 is already committed**, we will test and commit **Task 2 only**.

Your `main.py` stays unchanged.

Task 2 — Exact Testing Procedure

Before testing, make sure your modified `game/game_engine.py` is saved.

Run:

```powershell
python main.py
```

Test 1 — Verify the timer reaches zero

1. Start the game.
2. Look at the top-right corner.

3. You should see something like:

```
Plain text
Time: 30s
```

4. Let the game run until the countdown reaches:

```
Plain text
Time: 0s
```

Expected result

The game should **stop normal gameplay** when the timer reaches zero.

Test 2 — Verify Game Over screen appears

Immediately after the timer reaches zero, the game window should change from the normal game screen to something similar to:

```
Plain text
GAME OVER

Final Score: 5

Waiting for input...
```

Expected result

The Game Over screen must appear **inside the Pygame window**.

You should **not** have to look at the terminal to see the final score.

Test 3 — Verify final score is displayed

Before the timer reaches zero, try to get a few points.

For example:

```
Plain text
```

```
Score: 4
Time: 2s
```

Wait for the timer to finish.

The Game Over screen should show:

```
Plain text
Final Score: 4
```

Important

The number must match the score from immediately before Game Over.

For example:

```
Plain text
Gameplay:
Score: 7

Timer reaches 0

Game Over:
Final Score: 7
```

Test 4 — Verify normal clicking no longer changes the game

This is important.

Once the Game Over screen appears:

1. Click around the game window.
2. Try clicking where moles used to appear.
3. Click multiple times.
4. Watch the final score.

Expected

The score must remain unchanged.

For example:

```
Plain text
```

```
Final Score: 7

click
click
click
click

Final Score: 7
```

There should be **no new points**.

Also, no new moles should appear.

Test 5 — Verify the game waits for input

After Game Over appears:

1. Don't immediately close the window.
2. Leave it sitting for several seconds.
3. Observe the screen.

Expected

The Game Over screen should remain visible.

The program should **not**:

- crash
- return to the normal game
- start another round
- close automatically
- continue spawning moles
- continue counting down

The final score should remain visible.

This demonstrates that the game has entered a stable **Game Over state**.

Test 6 — Verify the game exits correctly

Your `main.py` already handles the Pygame close button:

```python
if event.type == pygame.QUIT:
```

```
        running = False
```

So while the Game Over screen is displayed:

1. Click the **X** button in the top-right of the Pygame window.
2. The window should close normally.
3. You should return to the VS Code terminal.

You should **not** get a Python traceback/error.

The terminal should simply return to your prompt, for example:

```
Plain text
PS C:\...\48_whack-a-mole>
```

Task 2 Pass Checklist

Before committing, verify all of these:

Test	Expected	Pass
Timer countdown	Reaches `0s`	☐
Game Over screen	Appears in Pygame window	☐
Final score	Correct score displayed	☐
Clicking after Game Over	Score doesn't change	☐
Game waits	Screen remains visible	☐
No new moles	Gameplay is stopped	☐
Close button	Window exits normally	☐
Terminal	No traceback/error	☐

If all are successful, Task 2 is ready to commit.

Task 2 Git Commit

First, stop the game if it's still running.

Then in the **VS Code terminal**, run:

1. Check what changed

```powershell
git status
```

You should see something similar to:

```
modified:   game/game_engine.py
```

You should **not** see unexpected modifications to other files.

---

2. Review the Task 2 changes

Run:

```powershell
git diff -- game/game_engine.py
```

Look through the changes.

You should see your Game Over implementation.

Since Task 1 was already committed, make sure you're not accidentally changing unrelated files.

---

3. Stage only `game_engine.py`

```powershell
git add game/game_engine.py
```

---

4. Verify the staged changes

powershell

```
git diff --cached
```

Make sure the staged code is the Task 2 change.

---

5. Create the Task 2 commit

Use your exact requested commit message:

powershell

```
git commit -m "Add game over screen and final score"
```

---

6. Verify the commit

powershell

```
git log -2 --oneline
```

You should see your Task 2 commit above your Task 1 commit, approximately:

Plain text

```
abc1234 Add game over screen and final score
def5678 Fix collision detection for single click
```

The actual hashes will obviously be different.

---

7. Finally check the working tree

powershell

```
git status
```

Ideally:

Plain text

```
nothing to commit, working tree clean
```

Final Task 2 state

You should now have two separate commits:

```
Plain text
```

```
Task 2
Add game over screen and final score
        ↓
Task 1
Fix collision detection for single click
```

Don't implement Task 3 yet. Once you have the clean `git status` and the two commits, send me the output of:

```
powershell
```

```
git status
git log -2 --oneline
```

and we'll verify Task 2 before moving on.

Task 2 is complete and committed.

Now I want to implement Task 3 only.

Task 3:
After Game Over, allow the user to play again by choosing:

1. Easy
2. Medium
3. Hard
4. Exit

Each difficulty should change the mole speed/spawn rate.

Please use my current Task 2 code and modify it.

Here is my current main.py:

```python
```

```
import pygame
from game.game_engine import GameEngine

pygame.init()

WIDTH, HEIGHT = 500, 560
SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Whack-a-Mole - Pygame Version")

GRASS_GREEN = (120, 170, 90)

clock = pygame.time.Clock()
FPS = 60

engine = GameEngine(WIDTH, HEIGHT)


def main():
    running = True

    while running:
        SCREEN.fill(GRASS_GREEN)

        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False

            elif engine.game_over and event.type ==
pygame.KEYDOWN:
                if event.key in (pygame.K_RETURN, pygame.K_SPACE):
                    running = False

            engine.handle_event(event)

        engine.handle_input()
        engine.update()
        engine.render(SCREEN)

        pygame.display.flip()
        clock.tick(FPS)
```

```
pygame.quit()


if __name__ == "__main__":
    main()
```

Yes. For **Task 3 only**, we need to change the Game Over behavior from:

```
GAME OVER
Final Score: X
Waiting for input...
```

to:

```
GAME OVER
Final Score: X

Choose Difficulty

1. Easy
2. Medium
3. Hard
4. Exit
```

The important part is that **Task 3 should not break Task 1 or Task 2**.

Difficulty behavior

We'll use:

Difficulty	Mole stays up	Spawn chance
Easy	60 frames ≈ 1.0 sec	0.015
Medium	45 frames ≈ 0.75 sec	0.020
Hard	30 frames ≈ 0.50 sec	0.030

So Hard means:

- moles disappear faster

- new moles appear more frequently

---

1. Replace `game/game_engine.py`

Replace your **entire current** `game/game_engine.py` with this:

Python

```
import pygame
import random
from .hole import Hole

# Game Engine

DARK_BROWN = (60, 40, 20)
MOLE_BROWN = (140, 95, 55)
BLACK = (0, 0, 0)
WHITE = (255, 255, 255)
DARK_GRAY = (40, 40, 40)


class GameEngine:
    def __init__(self, width, height, rows=3, cols=3):
        self.width = width
        self.height = height

        self.rows = rows
        self.cols = cols
```

---

2. Replace `main.py`

Your current `main.py` contains this Task 2 behavior:

Python

```
elif engine.game_over and event.type == pygame.KEYDOWN:
    if event.key in (pygame.K_RETURN, pygame.K_SPACE):
        running = False
```

That would incorrectly **exit the game when Enter/Space is pressed**, which we don't want anymore.

The GameEngine should now handle the difficulty choices itself.

Replace your complete `main.py` with:

Python

```
import pygame
from game.game_engine import GameEngine

pygame.init()

WIDTH, HEIGHT = 500, 560
SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Whack-a-Mole - Pygame Version")

GRASS_GREEN = (120, 170, 90)

clock = pygame.time.Clock()
FPS = 60

engine = GameEngine(WIDTH, HEIGHT)


def main():
    running = True
```

---

3. What changed for Task 3?

During normal gameplay

Nothing important changes.

Your Task 1 collision fix remains:

Python

```
if hole.whack():
    self.score += 1
    hit_something = True
    break
```

So the **one-click → maximum one point** behavior is preserved.

Your Task 2 timer remains:

Python

```
if self.time_left_frames <= 0:
    self.time_left_frames = 0
```

```
        self.game_over = True
```

So the Game Over state is preserved.

---

4. New Game Over menu

When the timer reaches zero, you'll now see:

```
Plain text
```

```
GAME OVER

Final Score: 12

Choose Difficulty

1. Easy
2. Medium
3. Hard
4. Exit

Press 1, 2, 3 or 4
```

---

5. Selecting Easy

Press:

```
Plain text
```

```
1
```

The current round ends and a **new round** starts.

Easy uses:

```
Python
```

```
spawn_chance = 0.015
mole_up_frames = 60
```

The score resets to:

Plain text

```
Score: 0
```

and the timer resets to:

Plain text

```
Time: 30s
```

---

6. Selecting Medium

Press:

Plain text

```
2
```

Medium uses:

Python

```
spawn_chance = 0.020
mole_up_frames = 45
```

This matches your original game's behavior.

---

7. Selecting Hard

Press:

Plain text

```
3
```

Hard uses:

Python

```
spawn_chance = 0.030
mole_up_frames = 30
```

So moles stay up for less time and spawning is more frequent.

---

8. Selecting Exit

Press:

```
Plain text
4
```

The game exits.

`Esc` also exits, although your required menu option remains **4. Exit**.

---

9. Important implementation detail

We reset the round through:

```
Python
start_new_round(difficulty)
```

This resets:

```
Plain text
score
timer
mole states
game_over state
difficulty
spawn rate
mole speed
```

So starting another difficulty is a **fresh round**, rather than continuing the previous score.

---

10. Test Task 3

Don't commit yet.

Run:

powershell

```
python main.py
```

Then test these cases.

Test A — Game Over menu

Let the timer reach zero.

Expected:

Plain text

```
GAME OVER
Final Score: X

Choose Difficulty

1. Easy
2. Medium
3. Hard
4. Exit
```

Test B — Easy replay

Press:

Plain text

```
1
```

Expected:

Plain text

```
Game Over disappears
        ↓
New game starts
        ↓
Score: 0
Time: 30s
Difficulty: Easy
```

Moles should stay visible relatively longer and spawn less frequently than Hard.

Test C — Medium replay

Let that round finish or restart the game and reach Game Over again.

Press:

```
Plain text
2
```

Expected:

```
Plain text
Score: 0
Time: 30s
Difficulty: Medium
```

Test D — Hard replay

At Game Over press:

```
Plain text
3
```

Expected:

```
Plain text
Score: 0
Time: 30s
Difficulty: Hard
```

Moles should disappear faster and spawn more frequently.

Test E — Exit

At Game Over press:

```
Plain text
```

```
4
```

Expected:

```
Plain text
Pygame window closes
Terminal returns
No traceback/error
```

Test F — Task 1 still works

During a replay, test your collision fix again.

One click on neighboring active moles must produce:

```
Plain text
+1 maximum
```

not:

```
Plain text
+2
```

This is important because Task 3 must not accidentally break Task 1.

Don't commit yet

Once you've tested everything, run:

```
powershell
git status
```

and:

```
powershell
git diff
```

Do not commit until we review the result.

Once Task 3 passes, we'll create its own separate commit with the message you choose, while keeping:

```
Plain text

Task 1
Fix collision detection for single click

Task 2
Add game over screen and final score

Task 3
[new commit]
```

separate.

Here is my current Task 2 game_engine.py:

```
import pygame
import random
from .hole import Hole

DARK_BROWN = (60, 40, 20)
MOLE_BROWN = (140, 95, 55)
BLACK = (0, 0, 0)



class GameEngine:
    def __init__(self, width, height, rows=3, cols=3):
        self.width = width
        self.height = height

        self.holes = []
        spacing_x = width // (cols + 1)
        spacing_y = (height - 80) // (rows + 1)

        for r in range(rows):
            for c in range(cols):
                cx = spacing_x * (c + 1)
                cy = 80 + spacing_y * (r + 1)
```

```
                self.holes.append(Hole(cx, cy))

        self.spawn_chance = 0.02
        self.mole_up_frames = 45

        self.round_seconds = 30
        self.time_left_frames = self.round_seconds * 60

        self.score = 0
        self.misses = 0
        self.font = pygame.font.SysFont("Arial", 28)
        self.game_over = False

    def handle_event(self, event):
        if self.game_over:
            return

        if event.type == pygame.MOUSEBUTTONDOWN:
            self._handle_click(event.pos)

    def _handle_click(self, pos):
        hit_something = False

        for hole in self.holes:
            if hole.rect().collidepoint(pos):
                if hole.whack():
                    self.score += 1
                    hit_something = True
                    break

        if not hit_something:
            self.misses += 1

    def handle_input(self):
        pass

    def update(self):
        if self.game_over:
            return
```

```
        self.time_left_frames -= 1

        if self.time_left_frames <= 0:
            self.game_over = True
            return

        for hole in self.holes:
            hole.update()

            if not hole.active and random.random() < self.spawn_chance
                hole.pop_up(self.mole_up_frames)

    def render(self, screen):
        for hole in self.holes:
            pygame.draw.circle(
                screen,
                DARK_BROWN,
                (hole.center_x, hole.center_y),
                40
            )

            if hole.active:
                pygame.draw.circle(
                    screen,
                    MOLE_BROWN,
                    (hole.center_x, hole.center_y),
                    32
                )

        score_text = self.font.render(
            f"Score: {self.score}",
            True,
            BLACK
        )
        screen.blit(score_text, (10, 10))

        seconds_left = max(0, self.time_left_frames // 60)

        timer_text = self.font.render(
            f"Time: {seconds_left}s",
```

```
            True,
            BLACK
        )
        screen.blit(
            timer_text,
            (self.width - 140, 10)
        )

        if self.game_over:
            overlay = pygame.Surface((self.width, self.height))
            overlay.set_alpha(180)
            overlay.fill((0, 0, 0))
            screen.blit(overlay, (0, 0))

            title_font = pygame.font.SysFont("Arial", 48)
            info_font = pygame.font.SysFont("Arial", 30)

            title = title_font.render("GAME OVER", True, (255, 255, 25
            score = info_font.render(
                f"Final Score: {self.score}",
                True,
                (255, 255, 255)
            )
            instruction = info_font.render(
                "Press ENTER or SPACE to exit",
                True,
                (255, 255, 255)
            )

            screen.blit(
                title,
                title.get_rect(center=(self.width // 2, 210))
            )
            screen.blit(
                score,
                score.get_rect(center=(self.width // 2, 280))
            )
            screen.blit(
                instruction,
```

```
            instruction.get_rect(center=(self.width // 2, 350))
        )
```

Yes — this is important. **Your actual Task 2 code is different from the version I previously assumed**, so we should build Task 3 directly on *this exact code*.

Also, your current `main.py` has:

```python
elif engine.game_over and event.type == pygame.KEYDOWN:
    if event.key in (pygame.K_RETURN, pygame.K_SPACE):
        running = False
```

For Task 3, that must be removed because **Enter/Space will no longer mean "exit"**. The Game Over screen will instead accept `1`, `2`, `3`, or `4`.

Task 3 design

We'll implement:

```
GAME OVER
Final Score: X

Choose Difficulty:

1. Easy
2. Medium
3. Hard
4. Exit
```

Difficulty settings

Difficulty	Spawn chance	Mole duration
Easy	0.015	60 frames
Medium	0.020	45 frames
Hard	0.030	30 frames

This means:

- **Easy:** slower/easier
- **Medium:** current/default behavior
- **Hard:** faster/more frequent

---

1. Replace `game/game_engine.py`

Use this complete version. It preserves your Task 1 collision fix and your Task 2 Game Over screen.

Python

```
import pygame
import random
from .hole import Hole

DARK_BROWN = (60, 40, 20)
MOLE_BROWN = (140, 95, 55)
BLACK = (0, 0, 0)


class GameEngine:
    def __init__(self, width, height, rows=3, cols=3):
        self.width = width
        self.height = height

        self.holes = []
        spacing_x = width // (cols + 1)
        spacing_y = (height - 80) // (rows + 1)

        for r in range(rows):
            for c in range(cols):
```

---

2. Replace `main.py`

Your current `main.py` contains the old Task 2 exit behavior:

Python

```
elif engine.game_over and event.type == pygame.KEYDOWN:
    if event.key in (pygame.K_RETURN, pygame.K_SPACE):
        running = False
```

Remove that completely.

Use this complete `main.py`:

Python

```
import pygame
from game.game_engine import GameEngine

pygame.init()

WIDTH, HEIGHT = 500, 560
SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Whack-a-Mole - Pygame Version")

GRASS_GREEN = (120, 170, 90)

clock = pygame.time.Clock()
FPS = 60

engine = GameEngine(WIDTH, HEIGHT)


def main():
    running = True
```

---

3. What happens now?

The flow is:

Plain text

```
Start game
    ↓
Medium difficulty
    ↓
Play for 30 seconds
    ↓
Timer = 0
    ↓
GAME OVER
    ↓
Final Score displayed
    ↓
Choose Difficulty
    │
    ├── 1 → Easy → new round
    │
    ├── 2 → Medium → new round
    │
    ├── 3 → Hard → new round
```

4. Test Task 3

Run:

```powershell
python main.py
```

Test 1 — Game Over menu

Let the timer reach zero.

You should see:

```
GAME OVER

Final Score: X

Choose Difficulty

1. Easy
2. Medium
3. Hard
4. Exit
```

Test 2 — Easy

Press:

```
1
```

Expected:

- Game Over disappears.
- New round starts.
- Score resets to `0`.
- Timer resets to `30`.
- Difficulty says `Easy`.

- Moles remain up for longer.
- Moles spawn less frequently.

Test 3 — Medium

Reach Game Over again and press:

```
Plain text
2
```

Expected:

- New round starts.
- Score = `0`.
- Timer = `30`.
- Difficulty = `Medium`.
- Original mole behavior is restored.

Test 4 — Hard

Reach Game Over and press:

```
Plain text
3
```

Expected:

- New round starts.
- Score = `0`.
- Timer = `30`.
- Difficulty = `Hard`.
- Moles disappear faster.
- Moles spawn more frequently.

Test 5 — Exit

Reach Game Over and press:

```
Plain text
```

```
4
```

Expected:

```
Plain text
```

```
Pygame window closes
↓
Terminal returns
↓
No traceback/error
```

Test 6 — Task 1 still works

During a replay, test the neighboring-mole case again.

One physical click must still give:

```
Plain text
```

```
+1 maximum
```

not:

```
Plain text
```

```
+2
```

Test 7 — Task 2 still works

Confirm:

- Timer reaches zero.
- Final score appears.
- Game stops.
- Game Over screen appears.
- Score doesn't change while at Game Over.

Don't commit yet

After testing, run:

powershell

```
git status
```

then:

powershell

```
git diff
```

Don't commit Task 3 yet.

Send me the test results (especially whether **1, 2, 3, and 4** all work). If everything passes, I'll give you the exact commands for the separate Task 3 commit.

> I tested the Task 3 implementation and found a problem during initialization.
>
> Please check the GameEngine.__init__ method carefully.
>
> Make sure time_left_frames is initialized immediately after round_seconds:
>
> ```python
> self.round_seconds = 30
> self.time_left_frames = self.round_seconds * 60
> ```

Yes. That initialization should be **immediately after** `self.round_seconds = 30` .

In your `GameEngine.__init__` , make sure this exact section is present:

Python

```
self.round_seconds = 30
self.time_left_frames = self.round_seconds * 60
```

So the relevant part should look like:

Python

```
self.spawn_chance = 0.02
self.mole_up_frames = 45
self.difficulty = "Medium"
```

```
self.round_seconds = 30
self.time_left_frames = self.round_seconds * 60

self.score = 0
self.misses = 0

self.font = pygame.font.SysFont("Arial", 28)

self.game_over = False
self.running = True
```

Why this matters

`time_left_frames` is used by:

```python
seconds_left = max(
    0,
    self.time_left_frames // 60
)
```

and by `update()`:

```python
self.time_left_frames -= 1
```

So it needs to exist as soon as the `GameEngine` object is created.

After making this correction

Save `game/game_engine.py` and run:

```powershell
python main.py
```

Then first verify that the game starts normally and shows:

```
Time: 30s
```

After that, continue with the Task 3 tests:

1. Let timer reach 0.
2. Verify Game Over menu.
3. Press `1` → Easy starts.
4. Reach Game Over again.
5. Press `2` → Medium starts.
6. Reach Game Over again.
7. Press `3` → Hard starts.
8. Reach Game Over again.
9. Press `4` → game exits.

Don't commit yet. This is still part of the Task 3 debugging/testing cycle.

Task 3 is now working.

I tested:
- Easy
- Medium
- Hard
- Exit

Please give me a proper testing checklist for Task 3 and then give me the exact Git commands to create a separate commit.

Use this commit message:

Add replay option and difficulty levels

Great. Since you have already tested **Easy, Medium, Hard, and Exit successfully**, we can now formally verify Task 3 and create its own commit.

Task 3 — Final Testing Checklist

Run:

```powershell
python main.py
```

1. Game Over appears

Let the 30-second timer reach zero.

Verify:

- [ ] Timer reaches `0s`
- [ ] Game Over screen appears
- [ ] Final score is displayed
- [ ] Difficulty menu is displayed

Expected:

```
Plain text
GAME OVER

Final Score: X

Choose Difficulty

1. Easy
2. Medium
3. Hard
4. Exit
```

---

2. Easy replay

At Game Over, press:

```
Plain text
1
```

Verify:

- [ ] Game Over screen disappears
- [ ] New round starts
- [ ] Score resets to `0`
- [ ] Timer resets to `30s`
- [ ] Difficulty shows `Easy`
- [ ] Moles remain up longer / spawn less frequently than Hard

---

3. Medium replay

Reach Game Over again and press:

```
Plain text
```

```
2
```

Verify:

- [ ] New round starts
- [ ] Score resets to `0`
- [ ] Timer resets to `30s`
- [ ] Difficulty shows `Medium`
- [ ] Medium uses the normal/default mole behavior

---

4. Hard replay

Reach Game Over again and press:

```
Plain text
3
```

Verify:

- [ ] New round starts
- [ ] Score resets to `0`
- [ ] Timer resets to `30s`
- [ ] Difficulty shows `Hard`
- [ ] Moles stay up for less time
- [ ] Moles spawn more frequently

---

5. Exit

Reach Game Over again and press:

```
Plain text
4
```

Verify:

- [ ] Pygame window closes
- [ ] Terminal returns normally
- [ ] No Python traceback/error appears

---

6. Verify Task 1 wasn't broken

During any replay:

- [ ] Click an active mole → +1
- [ ] Click an empty area → +0
- [ ] Test two neighboring active moles → **one click gives at most +1**

---

7. Verify Task 2 wasn't broken

- [ ] Timer reaches zero
- [ ] Game Over appears
- [ ] Correct final score appears
- [ ] Normal gameplay stops at Game Over
- [ ] No new moles spawn after Game Over

If all of these pass, **Task 3 is complete**.

Commit Task 3

First stop the game if it's still running.

Step 1 — Check status

```powershell
git status
```

You should see your Task 3 modifications, most likely:

```
modified:   game/game_engine.py
modified:   main.py
```

Step 2 — Review your changes

Run:

```powershell
git diff
```

Make sure the changes are related to:

- replay
- difficulty selection
- Easy/Medium/Hard settings
- Exit option
- Game Over input handling

Step 3 — Stage the Task 3 files

```powershell
git add game/game_engine.py main.py
```

Step 4 — Verify what will be committed

```powershell
git diff --cached
```

Make sure the staged changes are correct.

Step 5 — Create the Task 3 commit

Use your exact commit message:

```powershell
git commit -m "Add replay option and difficulty levels"
```

Step 6 — Verify your commit history

```powershell
git log -3 --oneline
```

You should see approximately:

Plain text

```
abc1234 Add replay option and difficulty levels
def5678 Add game over screen and final score
ghi9012 Fix collision detection for single click
```

The hashes will be different on your repository.

Step 7 — Confirm clean working tree

powershell

```
git status
```

The ideal result is:

Plain text

```
On branch main
nothing to commit, working tree clean
```

Your lab progress now

Plain text

```
✅ Task 1 — Fix collision detection
   Commit: Fix collision detection for single click

✅ Task 2 — Game Over screen
   Commit: Add game over screen and final score

✅ Task 3 — Replay + difficulty
   Commit: Add replay option and difficulty levels

⬜ Task 4 — Sound feedback
```

Do not implement Task 4 yet. Once you confirm the Task 3 commit is clean, we'll handle **only Task 4** as another separate Vibe Coding iteration.

Task 1, Task 2 and Task 3 are complete and committed separately.

Now I want to implement Task 4 only.

Task 4:
Add basic sound effects for:

1. Successful whack
2. Missed click
3. Round ending / Game Over

I want to use WAV files so the project does not depend on external music or copyrighted audio.

Please first explain the simplest approach.

Then tell me exactly which sound files need to exist and where they should be placed.

Do not modify the code yet until we decide how to create the sound files.

Yes. The simplest approach is to use **three short `.wav` sound-effect files** stored locally in your repository and load them with Pygame's `pygame.mixer.Sound()`.

Recommended structure

Add a `sounds` folder:

```
48_whack-a-mole/
│
├── main.py
├── requirements.txt
├── README.md
│
├── game/
│   ├── game_engine.py
│   └── hole.py
│
└── sounds/
    ├── whack.wav
    ├── miss.wav
    └── game_over.wav
```

The three files

File	When it plays
`sounds/whack.wav`	Successful hit on an active mole
`sounds/miss.wav`	Click that doesn't successfully hit a mole
`sounds/game_over.wav`	Timer reaches zero / Game Over

Keep them **short**—roughly 0.1–1 second each is enough.

Why WAV?

This is a good choice for your lab because:

- The files are stored locally.
- No internet connection is required during gameplay.
- No external music service is required.
- Pygame can play WAV files directly.
- You can use sounds you create yourself, avoiding copyright/licensing concerns.
- The sounds can be committed to GitHub along with the project.

Simplest implementation later

Once the files exist, we'll initialize Pygame's mixer and load:

```
</> Python

self.whack_sound = pygame.mixer.Sound("sounds/whack.wav")
self.miss_sound = pygame.mixer.Sound("sounds/miss.wav")
self.game_over_sound = pygame.mixer.Sound("sounds/game_over.wav")
```

Then the existing logic maps naturally:

```
</> Plain text

successful hole.whack()
        ↓
    whack.wav
```

```
</> Plain text

missed click
        ↓
     miss.wav
```

```
Plain text
timer reaches 0
        ↓
 game_over.wav
```

We **don't need background music**, volume controls, streaming audio, or any external audio library.

Before we modify code

We need to decide **how you're going to create the three WAV files**.

You have three good options:

1. **Create simple sounds yourself** — I can give you a small Python script that generates three basic WAV effects. This is the safest option for copyright/licensing.
2. **Use existing royalty-free/public-domain sound effects** — download three WAV files and put them in `sounds/` .
3. **Use a sound-generation tool** — generate the three effects and save them as WAV.

For a Software Engineering lab, **Option 1 is the cleanest and easiest to reproduce**: the project can contain a tiny sound-generation script, and nobody needs to download copyrighted assets.

If you choose that approach, we can create simple synthesized:

- `whack.wav`
- `miss.wav`
- `game_over.wav`

without changing your game code yet.

I want to generate simple WAV sound effects locally using Python so that I do not need to download external audio.

Please create a complete Python script named:

game/create_sounds.py

The script should generate these three files:

game/sounds/whack.wav
game/sounds/miss.wav
game/sounds/game_over.wav

Use Python's standard library only.

The sounds can be simple tones:
- whack: short higher-pitched sound
- miss: short lower-pitched sound
- game over: longer lower-pitched sound

Please give me the complete create_sounds.py code and the exact command to run it from the project root.

Analyzed

Yes. Use this as `game/create_sounds.py`. It uses **only Python standard-library modules** (`wave`, `math`, `struct`, `pathlib`) and creates the `sounds` directory automatically.

Python

```
import math
import struct
import wave
from pathlib import Path


SAMPLE_RATE = 44100
AMPLITUDE = 0.35


def generate_tone(filename, frequency, duration, volume=AMPLITUDE):
    """Generate a simple mono 16-bit WAV tone using only the standard library."""
    output_path = Path(__file__).resolve().parent / "sounds" / filename
    output_path.parent.mkdir(parents=True, exist_ok=True)

    num_samples = int(SAMPLE_RATE * duration)

    with wave.open(str(output_path), "wb") as wav_file:
        wav_file.setnchannels(1)
        wav_file.setsampwidth(2)  # 16-bit audio
```

Put it here

Your project should look like:

Plain text

```
48_whack-a-mole/
│
├── main.py
├── requirements.txt
├── README.md
│
└── game/
```

```
    ├── __init__.py
    ├── game_engine.py
    ├── hole.py
    └── create_sounds.py
```

You **don't need to manually create** `sounds/` . The script creates it.

Run it from the project root

Open the VS Code terminal at:

```
48_whack-a-mole>
```

Then run:

```
python game/create_sounds.py
```

You should get:

```
Created: ...\game\sounds\whack.wav
Created: ...\game\sounds\miss.wav
Created: ...\game\sounds\game_over.wav

All sound files generated successfully.
```

Afterward:

```
game/
├── game_engine.py
├── hole.py
├── create_sounds.py
└── sounds/
    ├── whack.wav
    ├── miss.wav
    └── game_over.wav
```

I've also created a copy of the script here if you want to use the generated file directly:
‹/› Download create_sounds.py

Don't modify `game_engine.py` yet. First run the generator and confirm that all three `.wav` files appear under `game/sounds/`. Then we'll integrate them into Task 4. ‹/›

‹/› **create_sounds.py**
Code

The three WAV files have been generated successfully.

Now I want to integrate them into the game.

Here is my current Task 3 game_engine.py:

```python
import pygame
import random
from .hole import Hole

DARK_BROWN = (60, 40, 20)
MOLE_BROWN = (140, 95, 55)
BLACK = (0, 0, 0)


class GameEngine:
    def __init__(self, width, height, rows=3, cols=3):
        self.width = width
        self.height = height

        self.holes = []
        spacing_x = width // (cols + 1)
        spacing_y = (height - 80) // (rows + 1)

        for r in range(rows):
            for c in range(cols):
                cx = spacing_x * (c + 1)
                cy = 80 + spacing_y * (r + 1)
                self.holes.append(Hole(cx, cy))

        self.spawn_chance = 0.02
```

```
        self.mole_up_frames = 45

        self.round_seconds = 30
        self.time_left_frames = self.round_seconds * 60

        self.score = 0
        self.misses = 0
        self.font = pygame.font.SysFont("Arial", 28)
        self.game_over = False

    def handle_event(self, event):
        if self.game_over:
            return

        if event.type == pygame.MOUSEBUTTONDOWN:
            self._handle_click(event.pos)

    def _handle_click(self, pos):
        hit_something = False

        for hole in self.holes:
            if hole.rect().collidepoint(pos):
                if hole.whack():
                    self.score += 1
                    hit_something = True
                    break

        if not hit_something:
            self.misses += 1

    def handle_input(self):
        pass

    def update(self):
        if self.game_over:
            return

        self.time_left_frames -= 1

        if self.time_left_frames <= 0:
```

```
                self.game_over = True
                return

            for hole in self.holes:
                hole.update()

                if not hole.active and random.random() <
        self.spawn_chance:
                    hole.pop_up(self.mole_up_frames)

        def render(self, screen):
            for hole in self.holes:
                pygame.draw.circle(
                    screen,
                    DARK_BROWN,
                    (hole.center_x, hole.center_y),
                    40
                )

                if hole.active:
                    pygame.draw.circle(
                        screen,
                        MOLE_BROWN,
                        (hole.center_x, hole.center_y),
                        32
                    )

            score_text = self.font.render(
                f"Score: {self.score}",
                True,
                BLACK
            )
            screen.blit(score_text, (10, 10))

            seconds_left = max(0, self.time_left_frames // 60)

            timer_text = self.font.render(
                f"Time: {seconds_left}s",
                True,
                BLACK
```

```
        )
        screen.blit(
            timer_text,
            (self.width - 140, 10)
        )

        if self.game_over:
            overlay = pygame.Surface((self.width, self.height))
            overlay.set_alpha(180)
            overlay.fill((0, 0, 0))
            screen.blit(overlay, (0, 0))

            title_font = pygame.font.SysFont("Arial", 48)
            info_font = pygame.font.SysFont("Arial", 30)

            title = title_font.render("GAME OVER", True, (255, 255, 255))
            score = info_font.render(
                f"Final Score: {self.score}",
                True,
                (255, 255, 255)
            )
            instruction = info_font.render(
                "1 Easy   2 Medium   3 Hard   4 Exit",
                True,
                (255, 255, 255)
            )

            screen.blit(
                title,
                title.get_rect(center=(self.width // 2, 210))
            )
            screen.blit(
                score,
                score.get_rect(center=(self.width // 2, 280))
            )
            screen.blit(
                instruction,
                instruction.get_rect(center=(self.width // 2, 350))
            )
```

I see one important issue before we integrate the sounds: **the code you pasted is not actually your working Task 3 version**.

Your pasted `game_engine.py` has:

```python
if self.game_over:
    return
```

inside `handle_event()`, and its Game Over screen only displays:

```
1 Easy   2 Medium   3 Hard   4 Exit
```

but it has **no code to handle keys 1–4**, no `start_new_round()`, no difficulty state, and no `running` state.

So if I replace this file with a sound-integrated version based on the pasted code, **we would accidentally remove your working Task 3 replay functionality**.

Don't change anything yet

Please paste your **actual working Task 3 `game_engine.py`** — the version you tested successfully with:

- `1` → Easy
- `2` → Medium
- `3` → Hard
- `4` → Exit

I specifically need the version containing something like:

```python
def _handle_game_over_input(...)
```

and:

```python
def start_new_round(...)
```

Once you paste that, I'll integrate exactly these three sounds **without changing Task 3 behavior**:

Plain text

```
Successful whack
    → game/sounds/whack.wav

Missed click
    → game/sounds/miss.wav

Timer reaches zero
    → game/sounds/game_over.wav
```

We'll also make sure each sound is triggered **exactly at the appropriate event**, rather than accidentally playing repeatedly every frame.

Your generated WAV files are already correct; we don't need to regenerate them.

Also tell me if main.py needs modification. If it does, provide the complete replacement main.py.

Task 4 code has been implemented.

Please give me the exact testing procedure.

I need to verify:

1. Successful whack plays whack.wav.
2. Missed click plays miss.wav.
3. Game Over plays game_over.wav.
4. Game Over sound plays only once.
5. Replay still works.
6. Easy, Medium and Hard still work.
7. Exit still works.
8. Tasks 1, 2 and 3 have not been broken.

Then give me the exact Git commands for the final separate Task 4 commit.

Use this commit message:

Add sound feedback for game events

Yes. Since you say **Task 4 code has already been implemented**, don't replace anything yet. We should first test the exact implementation you currently have.

Does `main.py` need modification?

Normally, no.

If your Task 4 implementation only initializes/loads the sounds and plays them from `GameEngine`, your existing Task 3 `main.py` should continue to work unchanged.

Your `main.py` only needs modification if your Task 4 implementation explicitly requires something from `main.py`, such as `pygame.mixer.init()` there.

The preferred structure is to initialize the mixer in the game startup and keep sound handling inside `GameEngine`. If your current Task 4 code already runs, **don't change `main.py` just for Task 4**.

So first test your current files as they are.

Task 4 — Exact Testing Procedure

From the project root:

```powershell
python main.py
```

Keep the game window and terminal visible during testing.

Test 1 — Successful whack → `whack.wav`

Procedure

1. Start the game.
2. Wait for a mole to appear.
3. Click directly on the active mole.
4. Listen for the sound.

Expected

You should hear the short, higher-pitched **whack** sound.

At the same time:

```
Score: X
```

should become:

```
Plain text
Score: X + 1
```

The mole should disappear/go down.

Verify

- [ ] `whack.wav` plays.
- [ ] Score increases by exactly 1.
- [ ] Mole disappears after successful hit.
- [ ] Sound doesn't require internet/external files.

Test 2 — Missed click → `miss.wav`

Procedure

During gameplay, click somewhere that does **not contain an active mole**.

For example:

```
Plain text
empty hole
```

or an area outside the mole.

Expected

You should hear the lower-pitched **miss** sound.

The score should **not increase**.

Example:

```
Plain text
Before:
Score: 5

Missed click
```

```
After:
Score: 5
```

Verify

- [ ] `miss.wav` plays.
- [ ] Score remains unchanged.
- [ ] No mole is incorrectly whacked.

Test 3 — Game Over → `game_over.wav`

Let the timer naturally reach:

```
Time: 0s
```

Expected

You should hear the longer, lower-pitched **Game Over** sound.

The Game Over screen should appear:

```
GAME OVER

Final Score: X

1 Easy   2 Medium   3 Hard   4 Exit
```

Verify

- [ ] Timer reaches zero.
- [ ] `game_over.wav` plays.
- [ ] Game Over screen appears.
- [ ] Final score is displayed.

Test 4 — Game Over sound plays ONLY ONCE

This is particularly important.

After Game Over appears, **wait for several seconds without pressing anything**.

Listen carefully.

Expected

You should hear:

```
Plain text

game_over.wav
      ↓
ONE TIME
```

It must **not** repeatedly play every frame.

For example, this is correct:

```
Plain text

Timer reaches 0
      ↓
game_over.wav 🔊
      ↓
GAME OVER screen remains
      ↓
No more game-over sounds
```

This would be wrong:

```
Plain text

Timer reaches 0
      ↓
🔊 🔊 🔊 🔊 🔊 🔊
```

Verify

- [ ] Sound plays once when the round ends.
- [ ] Waiting on Game Over doesn't replay it.
- [ ] Selecting a replay does not cause multiple Game Over sounds.

Test 5 — Replay still works

At the Game Over screen press:

```
Plain text
```

```
1
```

Expected

A new game starts.

Verify:

- [ ] Game Over screen disappears.
- [ ] Score resets to `0`.
- [ ] Timer resets to `30s`.
- [ ] Gameplay resumes.
- [ ] Sounds still work.

Test 6 — Easy / Medium / Hard

You need to verify that Task 4 didn't break Task 3.

Easy

Reach Game Over and press:

Plain text

```
1
```

Verify:

Plain text

```
Difficulty: Easy
```

and the game starts normally.

- [ ] Easy works.
- [ ] Moles use Easy timing/spawn settings.
- [ ] Whack sound works.
- [ ] Miss sound works.

Medium

Reach Game Over and press:

Plain text

```
2
```

Verify:

Plain text

```
Difficulty: Medium
```

- [ ] Medium works.
- [ ] New round starts.
- [ ] Sounds work.

Hard

Reach Game Over and press:

Plain text

```
3
```

Verify:

Plain text

```
Difficulty: Hard
```

- [ ] Hard works.
- [ ] New round starts.
- [ ] Moles use Hard timing/spawn settings.
- [ ] Sounds work.

Test 7 — Exit

Reach Game Over and press:

Plain text

```
4
```

Expected

The Pygame window should close.

You should return to your terminal:

```
Plain text
PS C:\...\48_whack-a-mole>
```

There should be no Python traceback.

Verify:

- [ ] `4` exits.
- [ ] Window closes normally.
- [ ] No error/traceback.

Test 8 — Verify Task 1 still works

This is important because Task 4 should not change your collision logic.

During gameplay, get two neighboring moles active if possible.

Make **one click** near their boundary.

Expected

One physical click gives:

```
Plain text
+1 maximum
```

Never:

```
Plain text
+2
```

Also verify:

- [ ] Active mole → +1.
- [ ] One click cannot produce +2.
- [ ] Missed click → +0.

- [ ] `whack.wav` only plays for a successful hit.
- [ ] `miss.wav` plays for a miss.

Test 9 — Verify Task 2 still works

Let the timer reach zero.

Verify:

```
Plain text
```

```
Time: 0s
        ↓
GAME OVER
        ↓
Final Score: X
```

Also verify:

- [ ] Final score is correct.
- [ ] Normal gameplay stops.
- [ ] No new moles spawn.
- [ ] Clicking during Game Over doesn't increase score.
- [ ] Game Over sound plays once.

Final Task 4 checklist

Before committing, you should be able to check everything:

```
Plain text
```

```
TASK 4
_____________________________________

☐ Successful whack → whack.wav
☐ Missed click → miss.wav
☐ Timer reaches zero → game_over.wav
☐ Game Over sound plays only once
☐ Game Over screen still works
☐ Replay works
☐ Easy works
☐ Medium works
☐ Hard works
☐ Exit works
```

```
TASK 1
_____________________________________

☐ One click = maximum +1
☐ Neighboring moles don't produce +2

TASK 2
```

Final Task 4 Commit

Once **all tests pass**, stop the game.

1. Check Git status

```powershell
git status
```

You should expect changes such as:

```
modified:   game/game_engine.py
```

and the new sound files:

```
game/sounds/whack.wav
game/sounds/miss.wav
game/sounds/game_over.wav
```

If you also added `game/create_sounds.py`, that should appear too.

---

2. Review all changes

Run:

```powershell
git diff
```

For the new WAV files, Git won't show useful text differences, so also run:

```powershell
git status
```

Make sure you haven't accidentally modified unrelated files.

---

3. Stage the Task 4 files

If your Task 4 changes are in `game_engine.py` and the sound assets:

```powershell
git add game/game_engine.py game/sounds/whack.wav game/sounds/miss.wav game/sounds/game_over.wav
```

If `create_sounds.py` is part of what you want submitted with the project, also stage it:

```powershell
git add game/create_sounds.py
```

You can simply stage all intended Task 4 files together:

```powershell
git add game/game_engine.py game/create_sounds.py game/sounds/
```

---

4. Verify the staged files

```powershell
git status
```

You should see your Task 4 files under:

```Plain text
Changes to be committed:
```

Then:

powershell

```
git diff --cached --stat
```

This gives you a quick summary of what will be committed.

---

5. Create the Task 4 commit

Use your exact commit message:

powershell

```
git commit -m "Add sound feedback for game events"
```

---

6. Verify the four commits

Run:

powershell

```
git log --oneline -4
```

You should have approximately:

Plain text

```
xxxxxxx Add sound feedback for game events
xxxxxxx Add replay option and difficulty levels
xxxxxxx Add game over screen and final score
xxxxxxx Fix collision detection for single click
```

The hashes will be different.

---

7. Verify clean repository

Finally:

powershell

```
git status
```

Ideally:

```
Plain text

On branch main
nothing to commit, working tree clean
```

Your four-task history

You should now have a clean progression:

```
Plain text

Task 1
Fix collision detection for single click
        ↓
Task 2
Add game over screen and final score
        ↓
Task 3
Add replay option and difficulty levels
        ↓
Task 4
Add sound feedback for game events
```

That gives you a very good **Vibe Coding lab history**, because each requirement is isolated in its own commit and can be demonstrated independently.

All four tasks are complete and tested.

I have also recorded the AFTER video showing the completed game.

The four Git commits are:

1. Fix collision detection for single click
2. Add game over screen and final score
3. Add replay option and difficulty levels
4. Add sound feedback for game events

The completed features are:

Task 1:

Single click cannot award multiple points because the click handling stops after the first successful whack.

Task 2:
A graphical Game Over screen displays the final score and waits for player input.

Task 3:
Replay is available with:
- Easy
- Medium
- Hard
- Exit

Task 4:
Sound feedback is available for:
- successful whack
- missed click
- Game Over

Please give me a final verification checklist before I copy the project into my personal GitHub repository.

Do not suggest pushing to the original SETAPESU26 repository.

Absolutely. Since **all 4 tasks are implemented, tested, committed, and the AFTER video is recorded**, this is the final pre-copy verification.

Final Verification Checklist

1. Git commit history

Run:

powershell

```
git log --oneline -4
```

Verify the four commits appear in this order:

Plain text

```
Add sound feedback for game events
Add replay option and difficulty levels
Add game over screen and final score
Fix collision detection for single click
```

- [ ] Task 1 commit exists
- [ ] Task 2 commit exists
- [ ] Task 3 commit exists
- [ ] Task 4 commit exists

---

2. Working tree is clean

Run:

```powershell
git status
```

Expected:

```
nothing to commit, working tree clean
```

- [ ] No uncommitted changes
- [ ] No accidentally modified files
- [ ] No unnecessary temporary files

---

3. Project structure

Make sure your project contains the required files.

It should look approximately like:

```
48_whack-a-mole/
│
├── main.py
├── requirements.txt
├── README.md
│
└── game/
```

```
    ├── game_engine.py
    ├── hole.py
    ├── create_sounds.py
    │
    └── sounds/
        ├── whack.wav
        ├── miss.wav
        └── game_over.wav
```

- [ ] `main.py`
- [ ] `requirements.txt`
- [ ] `README.md`
- [ ] `game/game_engine.py`
- [ ] `game/hole.py`
- [ ] `game/create_sounds.py`
- [ ] `whack.wav`
- [ ] `miss.wav`
- [ ] `game_over.wav`

---

4. Task 1 verification

Collision detection

- [ ] Active mole can be successfully clicked.
- [ ] Successful click gives **exactly +1**.
- [ ] One click cannot give +2.
- [ ] Neighboring/overlapping mole hitboxes don't cause multiple points.
- [ ] Missed click gives no score.

Your implementation specifically stops after the first successful `whack()`.

---

5. Task 2 verification

Game Over

- [ ] Timer counts down normally.
- [ ] Timer reaches `0`.
- [ ] Game stops accepting normal gameplay clicks.
- [ ] Graphical `GAME OVER` screen appears.
- [ ] Final score is displayed.
- [ ] Game doesn't rely on a console `print()` for Game Over.

- [ ] No traceback occurs.

---

6. Task 3 verification

Replay menu

At Game Over:

- [ ] `1` → Easy
- [ ] `2` → Medium
- [ ] `3` → Hard
- [ ] `4` → Exit

For each difficulty:

- [ ] New round starts.
- [ ] Score resets.
- [ ] Timer resets.
- [ ] Difficulty changes correctly.
- [ ] Moles behave according to the selected difficulty.

---

7. Task 4 verification

Successful whack

- [ ] `whack.wav` plays.
- [ ] Score increases by 1.

Miss

- [ ] `miss.wav` plays.
- [ ] Score does not increase.

Game Over

- [ ] `game_over.wav` plays when the timer reaches zero.
- [ ] Game Over sound plays **only once**.
- [ ] Waiting on the Game Over screen doesn't repeatedly trigger the sound.

Replay + sound

- [ ] After replaying, whack sound still works.
- [ ] Miss sound still works.
- [ ] Next Game Over sound still works once.

8. AFTER video

You already recorded the AFTER video.

Make sure it demonstrates enough of the completed functionality for your submission:

- [ ] Successful whack
- [ ] Score increasing
- [ ] Miss behavior
- [ ] Game Over screen
- [ ] Final score
- [ ] Replay menu
- [ ] At least one difficulty selection
- [ ] Sound feedback is audible/visible through the recording if required

You **don't necessarily need to record another video** if your existing AFTER video already demonstrates the required completed behavior.

---

9. BEFORE video

Your lab requires the BEFORE video showing the original broken behavior.

Make sure you still have:

```
Plain text

BEFORE video
+
AFTER video
```

- [ ] BEFORE video preserved
- [ ] AFTER video preserved
- [ ] Both are around the required 10-second demonstration length
- [ ] BEFORE demonstrates the original bug/problem
- [ ] AFTER demonstrates the fixed/completed behavior

Don't delete the BEFORE video just because the project is now complete.

---

10. Check `requirements.txt`

Open:

Plain text

```
requirements.txt
```

Make sure the dependency required to run the game is present, e.g. Pygame.

Also make sure you haven't accidentally added unnecessary packages just for generating the sounds.

Your WAV generator uses Python's standard library, so you don't need an audio-generation package.

---

11. Check for unnecessary files

Before copying, look for things such as:

Plain text

```
__pycache__/
*.pyc
.vscode/
temporary screenshots
temporary test files
random WAV files
debug files
```

You don't want accidental development junk in the personal repository.

If you have a Python cache folder, for example:

Plain text

```
game/__pycache__/
```

it normally shouldn't be included in the repository.

---

12. Check `.gitignore`

If your project has `.gitignore`, make sure Python temporary/cache files are ignored.

For example:

Plain text

```
__pycache__/
*.pyc
.venv/
venv/
```

Don't ignore the actual sound files:

```
Plain text
```

```
game/sounds/*.wav
```

because the WAV files are part of your completed game.

---

13. Test from a fresh terminal

This is an especially useful final check.

Close your current terminal/game.

Open a new terminal in the project folder and run:

```
powershell
```

```
python main.py
```

Verify the game launches normally.

This catches situations where the current terminal/session happens to have something configured that a fresh environment doesn't.

- [ ] Fresh terminal launches game
- [ ] No missing-file error
- [ ] Sounds load
- [ ] Game works normally

---

14. Check sound paths

This is important before copying to another repository.

Your game should reference:

```
Plain text
game/sounds/whack.wav
game/sounds/miss.wav
game/sounds/game_over.wav
```

The sound files must actually be inside the project.

Don't rely on WAV files sitting somewhere else on your computer.

---

15. Final Git check

Run:

```
powershell
git status
```

Then:

```
powershell
git log --oneline -4
```

You want:

```
Plain text
working tree clean
```

and the four Task commits.

Final status

If all of these are checked, your lab project is ready to copy:

```
Plain text
✅ Task 1 complete
✅ Task 2 complete
✅ Task 3 complete
✅ Task 4 complete
✅ All features tested
```

- ✅ BEFORE video saved
- ✅ AFTER video saved
- ✅ Four separate commits
- ✅ Sound files included
- ✅ README present
- ✅ requirements.txt present
- ✅ No unnecessary files
- ✅ Git working tree clean

One important point

Since you specifically said **not to push to `SETAPESU26`**, don't run any command that pushes to the assigned repository.

Once the checklist is clean, you can copy the completed project into **your personal GitHub repository** and preserve these four commits/history if you want the development progression visible.

I am NOT supposed to push changes to the original SETAPESU26 repository.

My personal GitHub repository is:

https://github.com/koushikraj52/SE-PES1UG24CS657-

It already contains my previous lab work.

I want to add a new folder:

Lab-4

Inside it I want this structure:

```
Lab-4/
├── 48_whack-a-mole/
│   ├── game/
│   │   ├── sounds/
│   │   │   ├── game_over.wav
│   │   │   ├── miss.wav
│   │   │   └── whack.wav
│   │   ├── create_sounds.py
│   │   ├── game_engine.py
│   │   └── hole.py
│   ├── main.py
```

```
│   ├── README.md
│   └── requirements.txt
│
├── Before_Video.mp4
├── After_Video.mp4
├── Chatgpt Link
└── README.md
```

Please explain the safest way to copy the completed project and videos into my personal repository without affecting my existing SE LAB2 and SE LAB3 folders.

Give me exact Windows commands.

Yes. The safest approach is to **leave your completed Lab-4 project repository untouched** and copy its files into a new `Lab-4` folder inside your existing personal repository.

Do **not** change the remote of your completed project and do **not** push anything to `SETAPESU26`.

Your personal repository remains:

```
koushikraj52/SE-PES1UG24CS657-
```

Recommended approach

We'll use this structure:

Plain text

```
SE-PES1UG24CS657-/
├── SE LAB2/
├── SE LAB3/
├── Lab-4/
│   ├── 48_whack-a-mole/
│   │   ├── game/
│   │   │   ├── sounds/
│   │   │   │   ├── game_over.wav
│   │   │   │   ├── miss.wav
│   │   │   │   └── whack.wav
│   │   │   ├── create_sounds.py
│   │   │   ├── game_engine.py
│   │   │   └── hole.py
│   │   ├── main.py
│   │   ├── README.md
│   │   └── requirements.txt
│   │
│   ├── Before_Video.mp4
```

```
│   ├── After_Video.mp4
│   ├── Chatgpt Link
```

Important

Do not copy the `.git` folder from the `48_whack-a-mole` project.

That prevents the Lab-4 folder from becoming a nested Git repository.

Step 1 — Locate your two folders

You should have something like:

```
Plain text
C:\Users\<YourName>\...\48_whack-a-mole
```

and your personal repository:

```
Plain text
C:\Users\<YourName>\...\SE-PES1UG24CS657-
```

The first is your completed Lab-4 project.

The second is your personal GitHub repository containing Lab 2 and Lab 3.

Step 2 — First make sure your personal repository is clean

Open PowerShell.

Go to your personal repository:

```
powershell
cd "C:\PATH\TO\SE-PES1UG24CS657-"
```

Replace the path with your actual location.

Then:

```
powershell
```

```
git status
```

You want:

```
Plain text
nothing to commit, working tree clean
```

If it is NOT clean

Stop here.

Don't copy anything yet. If you have existing uncommitted Lab 2/3 work, we should protect that first.

Step 3 — Update your personal repository

Run:

```
powershell
git pull origin main
```

This makes sure your local personal repository matches your GitHub repository before adding Lab 4.

Then:

```
powershell
git status
```

Again, ideally:

```
Plain text
nothing to commit, working tree clean
```

Step 4 — Create the Lab-4 folder

From the root of your personal repository:

```
powershell
```

```
New-Item -ItemType Directory -Path ".\Lab-4"
```

Then verify:

powershell

```
Get-ChildItem
```

You should see:

Plain text

```
Lab-4
SE LAB2
SE LAB3
...
```

Your existing folders have **not been changed**.

Step 5 — Copy the completed `48_whack-a-mole`

Suppose your completed project is located at:

Plain text

```
C:\Users\YourName\Desktop\48_whack-a-mole
```

Run:

powershell

```
Copy-Item -Path "C:\Users\YourName\Desktop\48_whack-a-mole" -Destination ".\Lab-4\" -Recurse
```

Replace the source path with the **actual location** of your completed project.

After this, you should have:

Plain text

```
Lab-4/
└── 48_whack-a-mole/
    ├── game/
```

```
    ├── main.py
    ├── README.md
    └── requirements.txt
```

Step 6 — Check that `.git` wasn't copied

Run:

```powershell
Test-Path ".\Lab-4\48_whack-a-mole\.git"
```

You want:

```
False
```

If it says `True`

STOP.

Don't commit yet.

Delete only the nested `.git` folder:

```powershell
Remove-Item ".\Lab-4\48_whack-a-mole\.git" -Recurse -Force
```

Then verify again:

```powershell
Test-Path ".\Lab-4\48_whack-a-mole\.git"
```

It should say:

```
False
```

This is important because your **personal repository's** `.git` should control the entire repository.

Step 7 — Copy the videos

Assuming your videos are somewhere like:

Plain text

```
C:\Users\YourName\Desktop\Before_Video.mp4
C:\Users\YourName\Desktop\After_Video.mp4
```

Run:

powershell

```
Copy-Item "C:\Users\YourName\Desktop\Before_Video.mp4" ".\Lab-4\Before_Video.mp4"
```

Then:

powershell

```
Copy-Item "C:\Users\YourName\Desktop\After_Video.mp4" ".\Lab-4\After_Video.mp4"
```

Replace the source paths with their actual locations.

Step 8 — Create `Chatgpt Link`

Your structure says:

Plain text

```
Lab-4/
└── Chatgpt Link
```

If this is intended to be a text file containing your ChatGPT conversation link, create it with:

powershell

```
New-Item -ItemType File -Path ".\Lab-4\Chatgpt Link"
```

Then open it:

```powershell
notepad ".\Lab-4\Chatgpt Link"
```

Paste your ChatGPT conversation link into it.

Save and close Notepad.

Important

Don't put your link directly into a Git command or accidentally include unrelated private information. Just put the required conversation URL in this file.

Step 9 — Create the Lab-4 README

Create:

```powershell
New-Item -ItemType File -Path ".\Lab-4\README.md"
```

Open it:

```powershell
notepad ".\Lab-4\README.md"
```

You can put something like:

```Markdown
# Software Engineering Lab 4

## Lab Title
VibeCoding - Whack-a-Mole Game

## Project
48_whack-a-mole

## Tasks Completed

### Task 1 - Collision Detection
Fixed the collision detection so that one click cannot award multiple points.
```

```
### Task 2 - Game Over
Added a graphical Game Over screen displaying the final score.

### Task 3 - Replay and Difficulty
Added replay options:
- Easy
- Medium
```

Save it.

Step 10 — Verify the complete structure

From the root of your personal repository run:

```powershell
tree .\Lab-4 /F
```

You should see approximately:

```
Lab-4
│   After_Video.mp4
│   Before_Video.mp4
│   Chatgpt Link
│   README.md
│
└───48_whack-a-mole
    │   main.py
    │   README.md
    │   requirements.txt
    │
    └───game
        │   create_sounds.py
        │   game_engine.py
        │   hole.py
        │
        └───sounds
                game_over.wav
                miss.wav
                whack.wav
```

This is the most important visual check before committing.

Step 11 — Check Git status

Now:

```powershell
git status
```

You should see **only your new Lab-4 files** as untracked/added.

You should **NOT** see things like:

```
modified: SE LAB2/...
modified: SE LAB3/...
```

If you see Lab 2 or Lab 3 modifications that you did not intentionally make:

STOP.

Don't run `git add .` or commit.

Tell me what `git status` shows.

Step 12 — Check exactly what Git sees

Before staging anything, run:

```powershell
git status --short
```

You should see entries beginning with:

```
?? Lab-4/
```

That's good.

It means Git sees the new folder without reporting changes to your existing labs.

Step 13 — Stage ONLY Lab-4

This is safer than `git add .`.

Run:

```powershell
git add .\Lab-4
```

Then:

```powershell
git status
```

You should see:

```
Changes to be committed:
    new file: Lab-4/...
```

and nothing from Lab 2 or Lab 3.

Step 14 — Verify the staged files

Run:

```powershell
git diff --cached --stat
```

This lets you see what is about to be committed.

You should see your Lab-4 project files and videos.

Important: MP4 files

Your videos may be large.

Check their sizes:

```powershell
Get-ChildItem ".\Lab-4\*.mp4" | Select-Object Name, Length
```

If the videos are very large, **don't commit yet**.

GitHub has file-size limitations, so tell me the output before proceeding.

If they're reasonably sized, we can continue.

Step 15 — Commit Lab 4 to your personal repository

Only after verifying everything above:

```powershell
git commit -m "Add Software Engineering Lab 4"
```

This commit belongs to your **personal repository**.

It does not modify the original `SETAPESU26` repository.

Step 16 — Final safety check

Run:

```powershell
git status
```

Then:

```powershell
git log --oneline -3
```

Your newest personal-repository commit should be:

```Plain text
```

```
Add Software Engineering Lab 4
```

Step 17 — Push ONLY to your personal repository

First verify your remote:

```powershell
git remote -v
```

You should see your personal repository:

```
https://github.com/koushikraj52/SE-PES1UG24CS657-.git
```

If it shows your personal repository

Then:

```powershell
git push origin main
```

If it shows **SETAPESU26**

DO NOT PUSH.

Stop and show me the output of:

```powershell
git remote -v
```

We'll correct the remote before pushing.

One important distinction

Your completed `48_whack-a-mole` project has its own four commits:

```
```

```
Fix collision detection for single click
Add game over screen and final score
Add replay option and difficulty levels
Add sound feedback for game events
```

When you copy the **files** into your personal repository's `Lab-4` folder, those four commits do **not** automatically become commits in the personal repository.

That's okay.

Your personal repository will have a new commit such as:

Plain text

```
Add Software Engineering Lab 4
```

while the actual completed project files are preserved under:

Plain text

```
Lab-4/48_whack-a-mole/
```

This is the safest approach because **your existing SE LAB2 and SE LAB3 directories remain untouched** and you don't accidentally turn `48_whack-a-mole` into a nested Git repository.

Please create the final Lab-4/README.md for my submission.

Student:

Name: B Koushikraj
SRN: PES1UG24CS657

Project:
Whack-a-Mole Game

The README should contain:

1. Student Details
2. Objective
3. Project
4. Tasks Completed
5. Technologies Used

6. Project Structure

Use the following task descriptions:

Task 1 - Refine Collision Detection

Improved the click-hit handling so that a single mouse click cannot award multiple points when neighboring mole hit-boxes overlap. The game stops checking after the first successful whack.

Task 2 - Game Over Condition

Added a graphical Game Over screen that displays the final score when the round timer reaches zero. The game waits for the player to choose the next action instead of only printing the final score in the console.

Task 3 - Replay Option

Added a replay system after Game Over with three difficulty levels:

- Easy
- Medium
- Hard

Each difficulty has a different mole spawn rate and mole visibility duration. The game state is reset when starting a new round. An exit option is also provided.

Task 4 - Sound Feedback

Added sound effects for:

- Successful mole whack
- Missed click
- Game Over

The project structure should be:

Lab-4/

```
├── 48_whack-a-mole/
│   ├── game/
│   │   ├── game_engine.py
│   │   ├── hole.py
│   │   ├── create_sounds.py
│   │   └── sounds/
│   │       ├── whack.wav
│   │       ├── miss.wav
│   │       └── game_over.wav
│   ├── main.py
│   ├── README.md
│   └── requirements.txt
│
├── Before_Video.mp4
├── After_Video.mp4
├── Chatgpt Link
└── README.md
```

Give me the complete Markdown content so I can directly paste it into README.md.

Here is the final submission-ready `Lab-4/README.md` . You can copy it directly into your file.

Lab-4 README.md

Software Engineering Lab 4

1. Student Details

Field	Details
Name	B Koushikraj
SRN	PES1UG24CS657

2. Objective

The objective of this lab is to use an LLM as a debugging and pair-programming partner to identify problems, implement improvements, test the changes, and iteratively enhance a partially working **Whack-a-Mole** Pygame application.

The project was developed incrementally, with each task implemented, tested, and committed separately.

---

3. Project

Whack-a-Mole Game

The project is a Pygame-based Whack-a-Mole game featuring:

- A 3 × 3 grid of mole holes
- Randomly appearing moles
- Mouse-based mole interaction
- Score tracking
- Countdown timer
- Game Over screen
- Replay and difficulty selection
- Sound feedback for game events

---

4. Tasks Completed

Task 1 - Refine Collision Detection

Improved the click-hit handling so that a single mouse click cannot award multiple points when neighboring mole hit-boxes overlap. The game stops checking after the first successful whack.

Task 2 - Game Over Condition

Added a graphical Game Over screen that displays the final score when the round timer reaches zero. The game waits for the player to choose the next action instead of only printing the final score in the console.

Task 3 - Replay Option

Added a replay system after Game Over with three difficulty levels:

- **Easy**
- **Medium**
- **Hard**

Each difficulty has a different mole spawn rate and mole visibility duration. The game state is reset when starting a new round. An exit option is also provided.

Task 4 - Sound Feedback

Added sound effects for:

- Successful mole whack
- Missed click
- Game Over

The sound effects are stored locally in the project's `sounds` directory.

---

5. Technologies Used

- **Python**
- **Pygame**
- **Git**
- **GitHub**
- **LLM-assisted debugging and pair programming**
- **WAV audio files**

---

6. Project Structure

```
Lab-4/
├── 48_whack-a-mole/
│   ├── game/
│   │   ├── game_engine.py
│   │   ├── hole.py
│   │   ├── create_sounds.py
│   │   └── sounds/
│   │       ├── whack.wav
│   │       ├── miss.wav
│   │       └── game_over.wav
│   ├── main.py
│   ├── README.md
│   └── requirements.txt
│
├── Before_Video.mp4
├── After_Video.mp4
├── Chatgpt Link
└── README.md
```

Description of Important Files

- `main.py` — Entry point of the Pygame application.
- `game_engine.py` — Contains the main game logic, collision handling, timer, score, Game Over, replay, difficulty, and sound functionality.
- `hole.py` — Defines the behavior and state of individual mole holes.
- `create_sounds.py` — Generates the local WAV sound effects used by the game.
- `sounds/` — Contains the sound files for successful whacks, missed clicks, and Game Over.
- `requirements.txt` — Contains the Python dependencies required to run the project.
- `Before_Video.mp4` — Demonstrates the original/broken behavior before modifications.
- `After_Video.mp4` — Demonstrates the completed game after implementing all four tasks.
- `Chatgpt Link` — Contains the link to the LLM conversation used during the development process.

Completed Features

The final version of the Whack-a-Mole game successfully includes:

- Single-click collision handling
- Accurate score calculation
- Graphical Game Over screen
- Final score display
- Replay functionality
- Easy, Medium, and Hard difficulty levels
- Exit option
- Successful-whack sound
- Missed-click sound
- Game Over sound
- Countdown timer
- Random mole spawning

All four assigned tasks were implemented, tested, and committed separately during development.

Everything has now been completed.

Please give me one final Lab 4 submission checklist.

I need to verify:

- [ ] Before video is present
- [ ] After video is present

- [ ] Completed 48_whack-a-mole project is present
- [ ] Task 1 is implemented
- [ ] Task 2 is implemented
- [ ] Task 3 is implemented
- [ ] Task 4 is implemented
- [ ] Four separate Git commits were made
- [ ] ChatGPT conversation link contains the complete VibeCoding interaction
- [ ] Chatgpt Link file exists in Lab-4
- [ ] README.md exists in Lab-4
- [ ] Existing SE LAB2 and SE LAB3 folders were not modified
- [ ] Changes were pushed only to my personal repository

Also tell me what screenshots or GitHub checks I should do before final submission.

Final Software Engineering Lab 4 Submission Checklist

You're at the final verification stage. **Do not make any more code changes unless one of these checks fails.**

1. Lab-4 folder contents

Open your personal repository locally and verify:

```
Plain text
```

```
Lab-4/
├── 48_whack-a-mole/
│   ├── game/
│   │   ├── game_engine.py
│   │   ├── hole.py
│   │   ├── create_sounds.py
│   │   └── sounds/
│   │       ├── game_over.wav
│   │       ├── miss.wav
│   │       └── whack.wav
│   ├── main.py
│   ├── README.md
│   └── requirements.txt
│
├── Before_Video.mp4
├── After_Video.mp4
├── Chatgpt Link
└── README.md
```

Checklist

- [ ] Before video is present
- [ ] After video is present
- [ ] Completed `48_whack-a-mole` project is present
- [ ] `game_engine.py` is present
- [ ] `hole.py` is present
- [ ] `create_sounds.py` is present
- [ ] All 3 `.wav` files are present
- [ ] `main.py` is present
- [ ] `requirements.txt` is present
- [ ] Project `README.md` is present

---

2. Verify the four tasks

Task 1 — Collision Detection

- [ ] One click cannot award multiple points
- [ ] Successful whack gives exactly +1
- [ ] Neighboring mole hitboxes cannot cause +2

Task 2 — Game Over

- [ ] Timer reaches zero
- [ ] Graphical Game Over screen appears
- [ ] Final score is displayed
- [ ] Normal gameplay stops
- [ ] Game waits for player input

Task 3 — Replay

- [ ] Easy works
- [ ] Medium works
- [ ] Hard works
- [ ] Exit works
- [ ] New round resets score
- [ ] New round resets timer
- [ ] Difficulty changes mole behavior

Task 4 — Sound

- [ ] Successful whack plays `whack.wav`
- [ ] Missed click plays `miss.wav`

- [ ] Game Over plays `game_over.wav`
- [ ] Game Over sound plays only once
- [ ] Sounds still work after replay

---

3. Verify the four Git commits

Inside the original completed project, you should have:

```
Plain text
Fix collision detection for single click
Add game over screen and final score
Add replay option and difficulty levels
Add sound feedback for game events
```

If you still have access to that project folder, run:

```
powershell
git log --oneline -4
```

Verify all four commits exist.

- [ ] Task 1 commit exists
- [ ] Task 2 commit exists
- [ ] Task 3 commit exists
- [ ] Task 4 commit exists

---

4. Verify the ChatGPT conversation

Open your ChatGPT conversation used for the VibeCoding work.

Check that it contains the **complete interaction**, including:

- [ ] Original project/problem
- [ ] Task 1 discussion
- [ ] Task 1 debugging/fix
- [ ] Task 2 discussion
- [ ] Task 2 implementation/testing
- [ ] Task 3 discussion
- [ ] Task 3 implementation/testing

- [ ] Task 4 discussion
- [ ] Sound generation/integration
- [ ] Testing
- [ ] Final completion

Your `Chatgpt Link` file should contain the URL to that conversation.

---

5. Verify `Chatgpt Link`

Inside:

```
Plain text
Lab-4/Chatgpt Link
```

open the file and click/copy the link.

- [ ] File exists
- [ ] Link is present
- [ ] Link opens the correct ChatGPT conversation
- [ ] Conversation contains the complete VibeCoding interaction

Don't just check that the file exists — actually open the link once.

---

6. Verify Lab-4 README

Open:

```
Plain text
Lab-4/README.md
```

Check that it contains:

- [ ] Student name: B Koushikraj
- [ ] SRN: PES1UG24CS657
- [ ] Objective
- [ ] Project
- [ ] Tasks Completed
- [ ] Technologies Used
- [ ] Project Structure

7. Most important: verify Lab 2 and Lab 3 weren't modified

From your personal repository root:

```powershell
git status
```

If your Lab-4 work has already been committed, ideally you should see:

```
nothing to commit, working tree clean
```

Then inspect your latest commit:

```powershell
git show --stat --oneline HEAD
```

Make sure the latest commit only contains Lab-4 files.

You should **not** see changes such as:

```
SE LAB2/...
SE LAB3/...
```

- [ ] SE LAB2 unchanged
- [ ] SE LAB3 unchanged
- [ ] No accidental files added to either folder

---

8. Verify the GitHub repository

Open your personal GitHub repository:

Your personal SE repository

Navigate to:

```
```

```
Lab-4/
```

Verify GitHub displays:

```
Plain text
```

```
Lab-4
├── 48_whack-a-mole
├── Before_Video.mp4
├── After_Video.mp4
├── Chatgpt Link
└── README.md
```

Then open:

```
Plain text
```

```
Lab-4/48_whack-a-mole/
```

and verify the complete project is visible.

---

9. Verify GitHub commit history

On your personal repository, open the **Commits** page.

Your Lab-4 addition should appear as your personal-repository commit.

Also remember:

The four VibeCoding commits belong to the original `48_whack-a-mole` Git history.

When you copied the project into your personal repository, you copied the project files; you did not automatically transfer its `.git` history.

That's fine unless your lab specifically requires the four commit history to be visible **inside your personal repository**.

---

10. Verify you pushed ONLY to your personal repository

From your personal repository:

```
powershell
```

```
git remote -v
```

It should point to:

```
Plain text
```

```
https://github.com/koushikraj52/SE-PES1UG24CS657-.git
```

- [ ] `origin` points to your personal repository
- [ ] No push was made to `SETAPESU26`
- [ ] Lab-4 appears on your personal GitHub repository

Screenshots I recommend taking

You don't need dozens of screenshots. Take a small set that provides strong evidence.

Screenshot 1 — GitHub repository

Show:

```
Plain text
```

```
SE-PES1UG24CS657-
├── SE LAB2
├── SE LAB3
└── Lab-4
```

This demonstrates that Lab 2 and Lab 3 remain alongside your new Lab 4.

Screenshot 2 — Lab-4 contents

Open `Lab-4` and show:

```
Plain text
```

```
48_whack-a-mole
Before_Video.mp4
After_Video.mp4
Chatgpt Link
README.md
```

Screenshot 3 — Project structure

Open:

Plain text

```
Lab-4/48_whack-a-mole/game/
```

Show the Python files and `sounds` folder.

Screenshot 4 — Git commits

If your instructor expects Git evidence, show the four commits:

Plain text

```
Fix collision detection for single click
Add game over screen and final score
Add replay option and difficulty levels
Add sound feedback for game events
```

Screenshot 5 — Final game

A screenshot of the completed game showing something useful such as:

Plain text

```
Score: ...
Time: ...
Difficulty: ...
```

Screenshot 6 — Game Over

Show:

Plain text

```
GAME OVER
Final Score: ...
1. Easy
2. Medium
3. Hard
4. Exit
```

This demonstrates Tasks 2 and 3 particularly well.

Final GitHub safety check

Before considering the submission finished, run these **three commands** from your personal repository:

```powershell
git status
```

```powershell
git remote -v
```

```powershell
git log --oneline -3
```

You want:

Status:

```
nothing to commit, working tree clean
```

Remote:

```
koushikraj52/SE-PES1UG24CS657-
```

Latest commit:

Your Lab-4 addition commit.

🟢 Final submission checklist

Copy this and tick everything:

```
SOFTWARE ENGINEERING LAB 4 — FINAL CHECK

PROJECT
```

```
☐ Completed 48_whack-a-mole project present
☐ main.py present
☐ game_engine.py present
☐ hole.py present
☐ create_sounds.py present
☐ requirements.txt present
☐ whack.wav present
☐ miss.wav present
☐ game_over.wav present

TASKS
☐ Task 1 — Collision Detection complete
☐ Task 2 — Game Over complete
☐ Task 3 — Replay/Difficulty complete
☐ Task 4 — Sound Feedback complete

VIDEOS
```

If **every box is checked**, your Lab 4 submission package is ready.

Today 8:25 AM



the other chat where we have discussed like professional

Worked for 7s

that chat we need upload as pdf

Worked for 8s